CHAPTER 11

Inference, Latency, Throughput and Cost

In this chapter

Introduction: The Cache That Made the Bill Go Up . . . 3
11.1 The Two-Term Latency Model . . . 4
11.2 Streaming: The Lever That Changes Perception, Not Physics . . . 5
11.3 Throughput Is a Different Objective . . . 6
11.4 The Cost Equation . . . 6
11.5 Where the Money Actually Goes: Two Shapes . . . 7
11.6 Prompt Caching: The Mechanics, the Economics, and the Trap . . . 8
11.7 The Batch API, and the Other Modifiers . . . 9
11.8 The Levers, in Order . . . 10
11.9 What the Model Excludes, and Why That Matters . . . 11
11.10 Where This Sits in the Architecture . . . 12
11.11 Code Walkthrough: Pricing a Request Honestly . . . 12
11.12 Mini Project: The Workload Economics Model . . . 14
11.13 Hands-on Labs . . . 17
11.14 Lab Solutions . . . 17
11.15 Best Practices . . . 18
11.16 Common Mistakes . . . 18
11.17 Performance Considerations . . . 20
11.18 Security Considerations . . . 21
11.19 Real Production Examples . . . 21
11.20 Certification Notes . . . 22
11.21 Practice Questions . . . 23
11.22 Answers and Explanations . . . 24
11.23 Ten Practical Exercises . . . 25
11.24 Five Challenge Labs . . . 26
11.25 Interview Questions . . . 26
11.26 Summary . . . 27
11.27 Glossary . . . 28
11.28 Further Reading . . . 29

Part 2 of 13 · Chapter 11 of 64. Next: Chapter 12 — Scaling Laws and Benchmarking.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Write down the **two-term latency model** — time-to-first-token plus output tokens times time-per-output-token — and say which term dominates a given workload.
2. Explain why **prefill and decode have different physics**, and why that makes input tokens cheap and output tokens expensive.
3. Compute the **cost of a request** from the published rate card, broken down by line item, and identify which line item dominates.
4. Explain **prompt caching's pricing mechanics** — the 1.25× and 2× write premiums and the 0.1× read — and derive the break-even point rather than quoting it.
5. Recognise the failure in which **caching is enabled and costs more than not caching at all**, and say what causes it.
6. Apply the **Batch API**, model selection, output trimming and caching in the right order for a given workload shape.
7. Distinguish **latency, throughput and cost** as three separate objectives, and say which levers move which.
8. Build a workload model that a finance conversation can be conducted from, and state honestly what it excludes.

NOTE

Prerequisites

Chapter 4, sections 4.9 and 4.10 — prefill and decode, the KV cache, and prompt caching's prefix-only mechanism. Everything commercial in this chapter is a consequence of that section. **Chapter 6** for tokens as the unit of cost, and for the tokenizer change that invalidates old counts. **Chapter 9** for the serving anatomy and the KV-cache memory arithmetic that explains why long context is priced as it is. **Chapter 10** for the tool-use and verification architecture whose token overhead we now price.

Chapter 9 told you what the model costs Anthropic to serve. This chapter tells you what it costs you to use, and — more usefully — which of the four or five available levers is the one your particular workload responds to.

Introduction: The Cache That Made the Bill Go Up

A team runs a support agent on Claude Opus 5. The prompt is substantial and mostly stable: a long system prompt, eight tool definitions, and a policy document — about 12,000 tokens before the customer says anything. At 50,000 requests a day the monthly bill is around $109,000, and somebody is asked to do something about it.

Prompt caching is the obvious answer, and it is the right answer. The team enables it, ships, and waits for the invoice.

The invoice goes **up**. Not by a rounding error — by about a fifth.

```
  configuration              per request    per month  vs no cache
```

```
caching off entirely          0.072750      109,125        0.0%
caching on, 0% hit rate       0.087750      131,625       20.6%
caching on, 95% hit rate      0.022200       33,300      -69.5%
```

The cause is one line near the top of the system prompt. Someone had added the current timestamp, so that the model would know the date — sensible advice, which this book gave in Chapter 10. That timestamp changes every request. Chapter 4's section 4.9 established that caching is **prefix-only**: because of causal masking, changing anything early invalidates every cache entry after it. So the prefix never matched, every request was a cache *write* at 1.25× the base input rate, and no request was ever a cache *read*.

The team had not misconfigured a feature. They had paid a 25% premium on 12,000 tokens, 50,000 times a day, to populate a cache that nothing ever read.

Move the timestamp to the end of the prompt and the third row happens: a 69.5% reduction, from $109,125 to $33,300 a month. Same feature, same model, same traffic. One line moved.

This chapter is about the arithmetic that makes that visible before the invoice does. There are only three quantities that matter — latency, throughput and cost — and each has a small number of levers. The difficulty is not that the levers are complicated. It is that **the right lever depends on the shape of your workload**, and most teams apply the lever they read about rather than the one their shape implies. So we will build a model, price two workloads with opposite shapes, and watch the same intervention be transformative for one and worthless for the other.

11.1 The Two-Term Latency Model

Chapter 4's section 4.9 split generation into two phases with genuinely different physics. That split is the whole latency model.

DEFINITION

TTFT, TPOT, and the two phases

Prefill processes the entire prompt in one parallel pass, computing and storing the keys and values for every position. It is compute-bound and uses the hardware efficiently. Its duration is **time to first token (TTFT)**.

Decode generates one token at a time, each step attending over the whole KV cache. It is strictly sequential and memory-bandwidth-bound. Its per-step duration is **time per output token (TPOT)**, sometimes called inter-token latency.

Total latency is therefore:

```
total  =  TTFT(prompt tokens)  +  output tokens × TPOT
```

Two consequences follow immediately, and both are counterintuitive until you have seen the numbers.

Input tokens are cheap and output tokens are expensive — in seconds as well as in dollars. Prefill handles thousands of tokens in one pass; decode handles one token per pass. A 12,000-token prompt and a 350-token answer are not remotely comparable amounts of work, and the answer is the larger one.

Decode dominates almost everything. Here is the model run on three scenarios, with a latency profile whose constants are illustrative and which you should replace with your own measurements — the conclusions do not depend on the constants:

```
profile: fixed 250 ms, prefill 90 ms/1k tok, decode 18 ms/token
scenario                            TTFT    total  decode share
agent, cold prefix 12.8k/350       1.40s    7.70s         81.8%
agent, cached prefix 12.8k/350     0.32s    6.62s         95.1%
report writer 1.8k/4000            0.41s   72.41s         99.4%
crossover: with a cold 12.8k prefix, decode overtakes TTFT at 78 output
tokens; with the prefix cached, at 18 tokens.
```

Even with a cold 12,800-token prompt, decode is 82% of the wall clock. Cache the prefix and it is 95%. For the report writer it is 99.4%, and the request takes over a minute almost entirely because it is generating four thousand tokens one at a time.

The crossover line is the number to remember. **Decode overtakes time-to-first-token at 78 output tokens on a cold 12.8k prompt, and at 18 tokens once the prefix is cached.** Essentially every real request generates more than that. So unless your workload is classification with one-word answers, latency optimisation means **generating fewer tokens**, and everything else is second-order.

VISUAL ANALOGY

Prefill is loading the lorry; decode is delivering the parcels

Loading a lorry is one operation on the whole load. Doubling the load does not double the time in the way you fear, because everything goes in together and the loading bay is designed for it. Delivering the parcels is one stop at a time, and doubling the number of parcels does double the day.

A dispatcher optimising the loading bay when the round has four hundred stops is optimising the wrong end of the day, and this is precisely what a team does when it spends a sprint shortening the system prompt while the model writes four-thousand-token reports.

Where the analogy breaks. Parcels can be delivered by two vans in parallel; the tokens of a single response cannot, because each one is conditioned on the last — that is what causal masking means. You can run many *requests* in parallel, which is throughput, but you cannot make one response come out faster by adding hardware. The analogy also understates the loading bay: prefill compute is quadratic in prompt length (Chapter 4, section 4.10), so a very long prompt does eventually become expensive to load, whereas a lorry does not get harder to fill as it gets fuller.

11.2 Streaming: The Lever That Changes Perception, Not Physics

If decode is 95% of a seven-second request, the user waits seven seconds. Unless you stream.

Streaming returns tokens as they are generated rather than buffering the whole response. Total generation time is unchanged — the physics is untouched — but the user sees the first words after TTFT rather than after TTFT plus decode. In the agent scenario above that is **0.32 seconds instead of 6.62**, a twenty-fold improvement in the number that determines whether an interface feels responsive.

This is the highest-return latency intervention available and it costs nothing, so it is worth being precise about what it does and does not do.

It does not reduce cost. The same tokens are generated and billed.

It does not reduce total time. If a downstream system needs the complete response before it can act — a validator, a parser, a database write — streaming buys that system nothing at all.

It changes the error surface. Once you have emitted tokens you cannot un-emit them, so a response that turns out to be malformed or that ends in a refusal has already been partly shown. Chapter 17 covers the streaming API and Chapter 28 covers the stop reasons you must handle at the end of a stream.

The architectural rule is straightforward: **stream to humans, buffer for machines.** And when a machine consumes the output, TTFT stops mattering and total time is the whole story — which puts you back to generating fewer tokens.

11.3 Throughput Is a Different Objective

Latency is how long one request takes. **Throughput** is how many you complete per unit time. They are not the same objective and they frequently trade against each other, which is the source of a great deal of confused engineering.

Chapter 9's section 9.19 explained the serving-side mechanism: continuous batching admits a new sequence the moment one finishes, and larger batches use the hardware more efficiently. Larger batches also mean any individual request waits longer. A serving stack tuned for maximum throughput will have worse per-request latency than one tuned for minimum latency, and there is no configuration that is best at both.

On the Claude API you do not tune the batching — Chapter 9's Figure 9.1 put the serving stack firmly in Anthropic's column. What you control is which side of the trade you ask for:

You want	Use	You give up
Lowest latency per request	Synchronous Messages API, streaming	Nothing much, but you pay full rate
Highest throughput per dollar	The Batch API	Synchronous response; results are asynchronous
Fastest possible output	Fast mode, where supported	A premium rate — see section 11.6

And what governs how many requests you may have in flight at all is **rate limits**, which are Chapter 19's subject. They are the reason a throughput plan is not simply "send more requests": the documentation describes Start, Build and Scale tiers with different limits, and a workload that exceeds them queues regardless of how much you are willing to spend.

11.4 The Cost Equation

Now the money. The structure is simple and the consequences are not.

```
cost = input_tokens × input_rate  +  output_tokens × output_rate
```

with input tokens split three ways once caching is involved — uncached input, cache writes, and cache reads — each at a different rate.

Here is the published rate card, verified against the official pricing documentation at the time of writing. All figures are USD per million tokens.

Model	Base input	5m cache write	1h cache write	Cache read	Output
Claude Fable 5	$10.00	$12.50	$20.00	$1.00	$50.00
Claude Opus 5	$5.00	$6.25	$10.00	$0.50	$25.00
Claude Sonnet 5 (to 31 Aug 2026)	$2.00	$2.50	$4.00	$0.20	$10.00
Claude Sonnet 5 (from 1 Sep 2026)	$3.00	$3.75	$6.00	$0.30	$15.00
Claude Haiku 4.5	$1.00	$1.25	$2.00	$0.10	$5.00

Four structural facts are visible in that table, and all four are examinable.

Output costs five times input, on every model. This ratio is remarkably stable across the family, and it is the single most useful thing in the table. It follows directly from section 11.1: decode is sequential and bandwidth-bound, so a generated token consumes far more of the serving system than a prefilled one.

The cache multipliers are constant ratios, not per-model rates: 1.25× base for a 5-minute write, 2× for a 1-hour write, 0.1× for a read. Learn the multipliers and you never need the absolute numbers.

Sonnet 5 is on introductory pricing until 31 August 2026, after which input and output both rise by 50%. If you are building a cost model in mid-2026, that is a scheduled 50% increase you should have in the spreadsheet rather than in the invoice. It is also a good illustration of why Chapter 9 insisted that volatile facts be marked and re-verified.

Long context does not carry a long-context premium. The documentation states that the full 1M-token window is available at standard pricing, and that a 900,000-token request is billed at the same per-token rate as a 9,000-token one. Caching and batch discounts apply across the full window. Long context is expensive because it is a lot of tokens, not because the tokens cost more — which is a different and more manageable problem.

11.5 Where the Money Actually Goes: Two Shapes

The cost equation is trivial. Applying it is not, because the answer to "what should we optimise?" depends entirely on the shape of the request. Here are two workloads from the same product, priced on Claude Opus 5.

```
support agent (big stable prefix, short answer)
  line item        per request    share     per month
  prefix              0.009450    42.6%        14,175
  variable_in         0.004000    18.0%         6,000
  output              0.008750    39.4%        13,125
```

```
  TOTAL               0.022200   100.0%        33,300
report writer (small prompt, long answer)
  line item        per request    share     per month
  prefix              0.000945     0.9%            57
  variable_in         0.003000     2.9%           180
  output              0.100000    96.2%         6,000
  TOTAL               0.103945   100.0%         6,237
```

The agent sends 12,000 stable tokens and gets 350 back. Even with caching at a 95% hit rate, the prefix is still its largest line item at 42.6%. **Caching is the lever.**

The report writer sends 1,800 tokens and generates 4,000. Output is 96.2% of its cost. Enabling prompt caching on this workload saves under 6% — measured, in the test suite — and no amount of prompt tightening will change that. **Output length is the lever, and after that, model choice.**

Same company, same rate card, opposite answers. A cost-reduction programme that applies caching everywhere will transform one endpoint and waste a quarter on the other, and — as the introduction showed — can make things actively worse if applied carelessly.

KEY TAKEAWAYS

The first question of any cost conversation

Not "which model?" and not "should we cache?" but: **what fraction of the bill is output tokens?**

Above roughly two thirds, you are in a generation-bound workload. Shorten the output, cap `max_tokens`, use a cheaper model, and consider whether the response needs to be prose at all.

Below roughly a third, you are in a prompt-bound workload. Cache the stable prefix, order the prompt so the stable part is genuinely first, and check your hit rate rather than assuming it.

You cannot answer this from the model. You answer it from your own `usage` fields, which every response carries.

11.6 Prompt Caching: The Mechanics, the Economics, and the Trap

Chapter 4 gave the mechanism. Here is the money.

What it does. A cached prefix skips prefill on subsequent requests, so you pay 0.1× the input rate for those tokens instead of 1×. It reduces time-to-first-token and input cost. It does nothing for decode, and nothing for output cost.

What it costs to populate. The first request pays a write premium: 1.25× base for a cache valid five minutes, 2× base for one valid an hour.

Where it breaks even. The documentation states that caching pays off after one cache read for the 5-minute duration and after two for the 1-hour duration. That is a claim we can derive rather than accept. For `n` requests over one cache lifetime, uncached costs `n × base`; cached costs `1 write + (n−1) reads`. Solving each case:

```
model                      5m write  1h write   5m n   1h n
Claude Opus 5                 1.25x     2.00x      2      3
Claude Sonnet 5               1.25x     2.00x      2      3
```

```
Claude Haiku 4.5               1.25x     2.00x      2      3
n = requests over one cache lifetime before caching is cheaper.
```

Two requests within five minutes, or three within an hour. The break-even is model-independent because the multipliers are ratios. This is an unusually low bar, which is why caching is nearly always worth enabling — **and why the failure mode is not "we didn't cache" but "we cached and it didn't work".**

The trap has one cause and it follows from causal masking. Because position `k`'s keys and values depend on positions 1 through `k`, changing anything early invalidates everything after it. So **any variable content near the start of the prompt destroys the entire cache**, and the request silently becomes a write instead of a read.

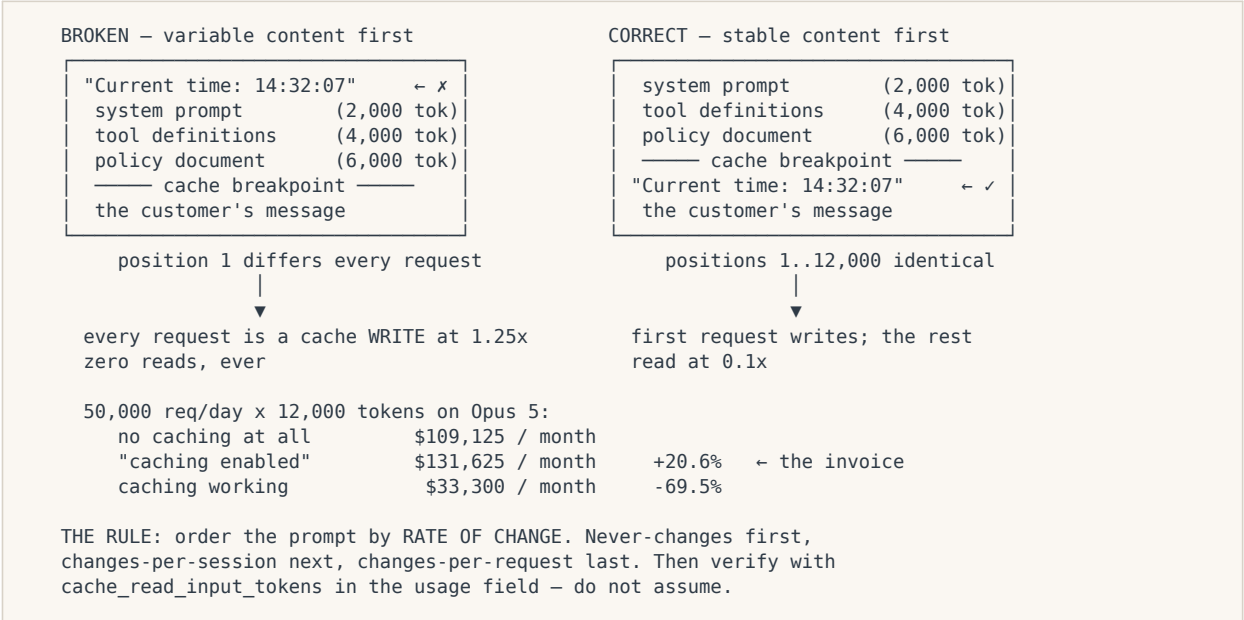


Figure 11.1 — Why one line at the top costs 20%.

Two further properties are worth carrying.

Cached tokens still occupy the context window. The documentation is explicit: prompt caching changes what you pay for those tokens, not whether they count. A cached 12,000-token prefix still consumes 12,000 tokens of your window and still contributes to the attention dilution of Chapter 4.

Caching and batching stack. The multipliers compose with the Batch API discount, so a cached, batched request pays 0.1× of the halved input rate.

11.7 The Batch API, and the Other Modifiers

Batch processing gives a **50% discount on both input and output tokens** in exchange for asynchronous processing. If your workload does not need a response while a human waits — overnight classification, backfills, evaluation runs, document processing, the regression suites of Chapter 24 — this is the single largest discount available and it requires no change to the prompt. Chapter 30 covers the mechanics.

Three other modifiers change the rate, and the exam expects you to know they exist.

Data residency. For Claude 4.6 and later, setting `inference_geo: "us"` applies a **1.1× multiplier to every token category** — input, output, cache writes and cache reads. Global routing is the default and is standard price. On partner-operated cloud platforms, regional and multi-region endpoints carry a 10% premium over global endpoints on the 4.5 generation and later.

Fast mode. In research preview at the time of writing, fast mode provides significantly faster output for Claude Opus 5 and Opus 4.8 at premium pricing — $10 input and $50 output per million tokens, double the standard rate. It is Claude API only, not available with the Batch API, and caching and data-residency multipliers stack on top of it. It is the one lever that buys decode speed directly, and it is priced accordingly.

Server-side tools carry their own charges. Web search is $10 per 1,000 searches plus the tokens of what it returns. Web fetch has no additional charge beyond the tokens of the fetched content — which the documentation notes can be substantial: roughly 2,500 tokens for an average web page and 125,000 for a research-paper PDF, which is why `max_content_tokens` exists. Code execution is free when used alongside web search or web fetch; used alone it is billed by container time, with 1,550 free hours per organisation per month and $0.05 per hour per container thereafter, and a five-minute minimum.

Tool definitions are input tokens, every request. Beyond the schemas you write, the API injects a tool-use system prompt: 286 tokens on Opus 5 with `tool_choice` of `auto` or `none`, 406 with `any` or `tool`; 354 and 474 on Sonnet 5; 496 and 588 on Haiku 4.5. Small individually, but they are on every request forever, and they belong in the cached prefix.

11.8 The Levers, in Order

Applying all of it to the agent workload from the introduction, one change at a time:

```
change                              per month      saved   cumulative
baseline: Opus 5, no caching          109,125          0         0.0%
+ prompt caching at 95%                33,300     75,825        69.5%
+ trim output 350 -> 200 tokens        27,675     81,450        74.6%
+ Sonnet 5 instead of Opus 5           16,605     92,520        84.8%
+ Batch API (async only)                8,302    100,823        92.4%
```

From $109,125 to $8,302 a month — a 92.4% reduction, with no change to the fundamental design.

Read that table carefully, because the ordering is not arbitrary and the last two rows are not free.

Caching first, because it is the largest single win on a prompt-heavy shape, requires no quality trade-off, and takes an afternoon. On the report-writer shape it would be first-and-worthless, which is the point of section 11.5.

Output trimming second, because output is billed at 5× input and is 95% of the latency. "Answer in at most three sentences" is a genuine engineering intervention. A cap on `max_tokens` is a genuine cost control. But this row *does* trade against quality, and the trade must be measured with Chapter 8's methodology rather than assumed.

Model selection third, and only after Chapter 10's compensating architecture has moved execution into tools — because once arithmetic, lookup and validation are outside the model, the remaining job is judgement and language, where the gap between tiers is often small. This row is a 50% saving on this

workload and it is the row most likely to be a mistake if taken on faith. Chapter 15 makes it a procedure.

Batch last, because it is not available at all if a human is waiting. It is a 50% discount with a hard architectural precondition, so it applies to whichever fraction of your traffic is genuinely asynchronous — and for many products that fraction is larger than anyone assumes.

Notice what is *not* on the list: reducing the variable input. It was 18% of the agent's cost and it is the part everyone edits first, because it is the part that feels like prompt engineering.

11.9 What the Model Excludes, and Why That Matters

A cost model that quietly omits a term is worse than no model, because it is trusted. This one omits five things, and each has caused a surprise.

Thinking tokens are output tokens. Per the documentation, thinking tokens are a subset of `max_tokens`, are billed as output, and count toward rate limits. With adaptive thinking the allocation varies per request, so your output token count becomes a distribution rather than a constant. A cost model built on the visible answer length will understate a thinking workload substantially.

Retries multiply everything. A request that fails and is retried is billed twice. A workload with a 3% error rate and an aggressive retry policy is a workload with a 3%-plus cost overhead. Chapter 29 covers retries and backoff properly.

Agentic loops multiply requests. Chapters 47 to 51 build systems where one user action produces a dozen model calls, each carrying the full conversation so far. The unit of cost stops being the request and becomes the *task*, and the growth is superlinear because the context accumulates. Model your agentic workloads per task, never per call.

Conversation history grows. In multi-turn conversation the input is not the user's latest message; it is everything so far. Cost per turn rises through a conversation even if every turn is short. This is Chapter 1's statelessness presenting its invoice, and Chapter 54's compaction is the answer.

The tokenizer changed. Chapter 6 verified it and the pricing documentation restates it: Claude 4.7 and later use a newer tokenizer producing approximately 30% more tokens for the same text, with the exact increase depending on content and workload shape. A cost model calibrated before that boundary is wrong after it by roughly a third. Re-measure with `count_tokens`, which is free.

> WARNING
>
> **The two numbers that make a cost model real**
>
> Everything above is arithmetic you can do in a spreadsheet. Two measurements turn it from a spreadsheet into a model.
>
> **The `usage` field on every response** reports what the request actually consumed, and when caching is on it splits input into `input_tokens`, `cache_read_input_tokens` and `cache_creation_input_tokens`. Log all three. Your **cache hit rate is `cache_read` divided by the sum**, and it is the single most valuable number in this chapter — because it is the difference between the second and third rows of the introduction's table, and nothing else in your telemetry will tell you which one you are in.
>
> **The `count_tokens` endpoint** estimates a request before you send it, is free, and is rate-limited independently of the Messages API. Use it to price a change before shipping it rather than after.

11.10 Where This Sits in the Architecture

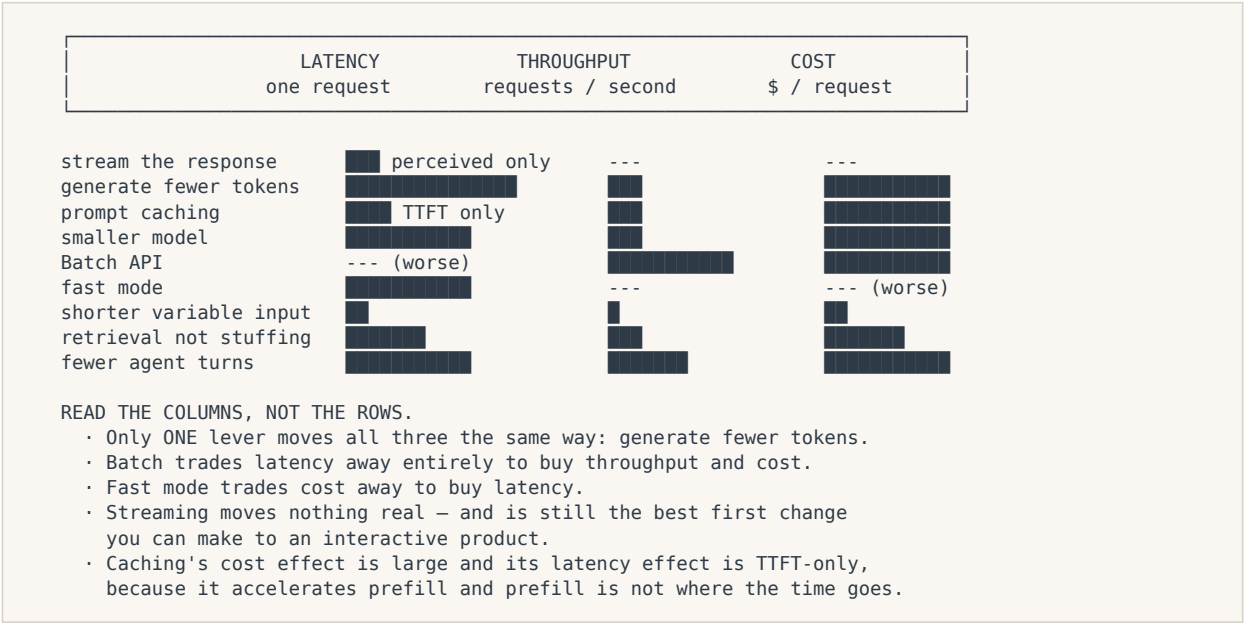


Figure 11.2 — The three objectives and the levers that move them.

The row worth internalising is the second. **Generating fewer tokens is the only lever that improves latency, throughput and cost simultaneously**, and it is the one that receives the least engineering attention because it looks like a product decision rather than a technical one. It is both. A system that answers in three sentences instead of three paragraphs is faster, cheaper and serves more users on the same rate limit — and, as Chapter 10 argued, is often better, because a shorter answer is a more checkable one.

11.11 Code Walkthrough: Pricing a Request Honestly

The whole cost model is about ninety lines. Its value is not cleverness — it is that every rate is transcribed from the published table and every derived claim is checked against the documentation's own prose.

Start with the rate card as a frozen dataclass, one row per model:

```
@dataclass(frozen=True)
class Pricing:
    """USD per million tokens. Columns as published."""
    name: str
    base_in: float          # standard input
    write_5m: float         # 5-minute cache write   (1.25x base)
    write_1h: float         # 1-hour cache write     (2x base)
    cache_read: float       # cache hit or refresh   (0.1x base)
    out: float              # output
```

`frozen=True` is deliberate. A rate card that can be mutated in place is a rate card that will be mutated in place — by a helper that "just applies the discount" — and you will spend an afternoon wondering why

the baseline moved. The modifiers instead return new objects:

```
    def batch(self) -> "Pricing":
        """The Batch API discounts input and output by 50%."""
        return replace(self, name=self.name + " (batch)",
                       base_in=self.base_in / 2, write_5m=self.write_5m / 2,
                       write_1h=self.write_1h / 2,
                       cache_read=self.cache_read / 2, out=self.out / 2)
    def us_only(self) -> "Pricing":
        """inference_geo='us' applies 1.1x to every token category."""
        return replace(self, name=self.name + " (us)",
                       base_in=self.base_in * 1.1, ...)
```

Both apply to **every** column, not just input and output. That matters: the documentation states the multipliers stack with each other and apply to cache writes and reads as well, so a batched, cached, US-pinned request is `0.1 × 0.5 × 1.1 × base`. Writing the modifiers as transformations of the whole rate card makes composition correct by construction rather than by remembering.

The workload is the other half of the model:

```
@dataclass
class Workload:
    """One request shape, repeated. Token counts are per request."""
    name: str
    requests_per_day: int
    stable_prefix: int       # system prompt, tool defs, reference documents
    variable_input: int      # the part that changes: the user's turn
    output: int              # generated tokens, including thinking
    cache_hit_rate: float = 0.0
    cache_ttl: str = "5m"
```

The single most important design decision in the module is that `stable_prefix` and `variable_input` are **separate fields**. Most cost calculators take one `input_tokens` number, and that number cannot express the difference between the second and third rows of the chapter's opening table — the difference between a $131,625 month and a $33,300 one. If your model cannot represent the mistake, it cannot warn you about it.

Note also that `output` is documented as including thinking tokens. That comment is doing real work: per the documentation, thinking tokens are billed as output, and a model that counts only the visible answer will understate a thinking workload badly.

The costing itself:

```
def cost_per_request(w: Workload, p: Pricing) -> dict:
    """Expected cost of one request, by line item, in USD."""
    write_rate = p.write_5m if w.cache_ttl == "5m" else p.write_1h
    hit, miss = w.cache_hit_rate, 1.0 - w.cache_hit_rate
    if w.cache_ttl is None:
        prefix_cost = w.stable_prefix * p.base_in / MTOK
    else:
        # A miss pays the write premium; a hit pays 0.1x.
        prefix_cost = w.stable_prefix * (
            miss * write_rate + hit * p.cache_read) / MTOK
    return {
        "prefix": prefix_cost,
        "variable_in": w.variable_input * p.base_in / MTOK,
```

```
        "output": w.output * p.out / MTOK,
    }
```

The expression `miss * write_rate + hit * p.cache_read` is the entire economics of prompt caching in one line, and it is why the broken configuration is expressible. Set `cache_hit_rate = 0.0` with caching enabled and `miss` is 1.0, so every request pays `write_rate` — which is 1.25× base, *more* than the `base_in` you would have paid with caching switched off. The failure is not a special case in the code. It is what the formula says when the hit rate is zero.

Returning a **dict of line items** rather than a single float is the other choice worth defending. A total tells you what to budget; a breakdown tells you what to fix. Section 11.5's entire argument — that the agent is prefix-dominated and the report writer is output-dominated — is invisible in a scalar.

Finally, the break-even, derived rather than quoted:

```
def cache_breakeven(p: Pricing, ttl: str = "5m") -> int:
    """Requests over one cache lifetime before caching is cheaper."""
    write = p.write_5m if ttl == "5m" else p.write_1h
    n = 1
    while write + (n - 1) * p.cache_read >= n * p.base_in:
        n += 1
        if n > 100:
            return -1
    return n
```

A closed form exists, and a loop is used anyway. The loop states the comparison in the same terms the documentation does — one write plus `n−1` reads against `n` uncached requests — so a reader can check the logic against the prose without doing algebra. The `n > 100` guard returns a sentinel rather than looping forever if a future rate card makes caching never pay off. Both choices are small; both are the difference between code you can audit and code you must trust.

11.12 Mini Project: The Workload Economics Model

What you are building. A model that takes a workload shape and a rate card and answers the four questions a cost conversation needs: what does a request cost by line item, where does caching break even, what does each lever save, and which latency term dominates. Plus the failure demonstration from the chapter's introduction, reproduced rather than described.

Why it is worth building. Because cost arguments are currently conducted with adjectives. "Opus is expensive." "We should cache." "Batch would help." Every one of those may be true or false for your workload, and the difference is one afternoon's arithmetic. The deliverable is not the code; it is the ability to walk into a review with a table.

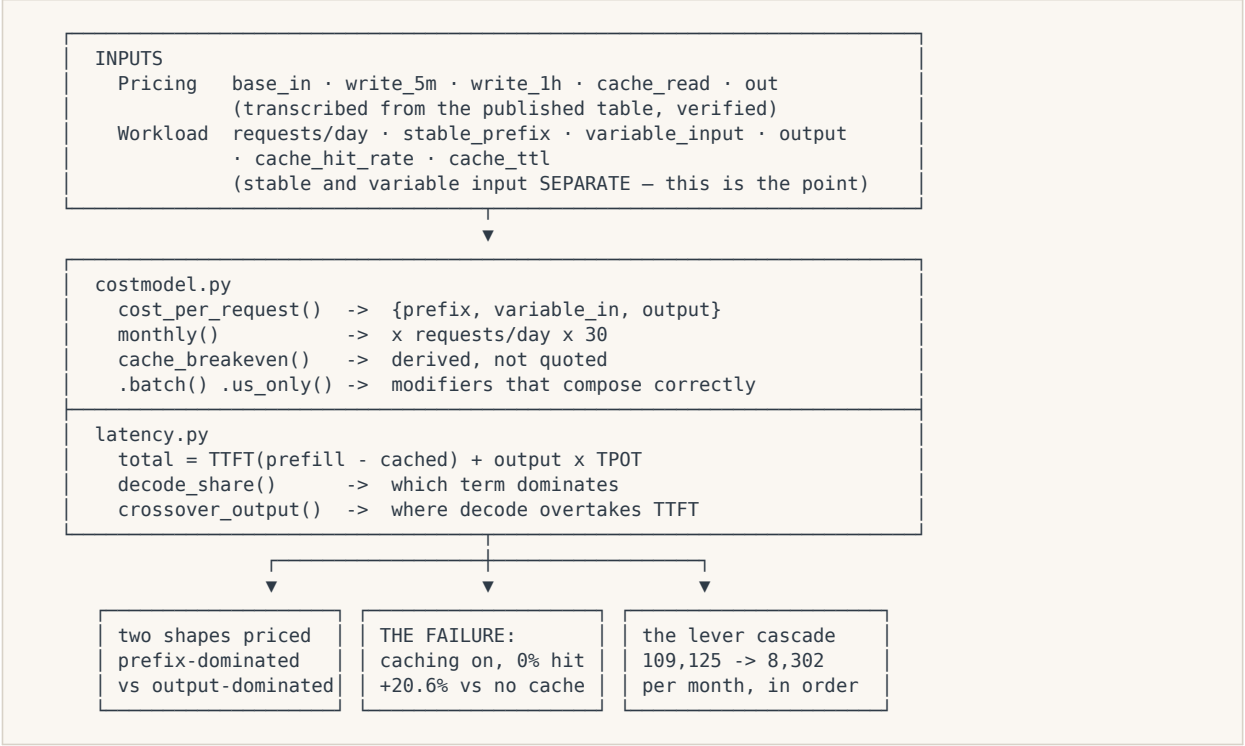


Figure 11.3 — The workload economics model.

Folder structure.

```
ch11/
  costmodel.py       rate card, workload, per-line-item costing, break-even
  latency.py         the two-term latency model and the crossover
  report.py          the five-section report reproduced in this chapter
  test_costmodel.py  12 tests, four validating against published figures
  test_latency.py    4 tests for the latency model's structural claims
```

The failure, reproduced. `report.py` builds the broken configuration by changing exactly one field, which is how the real incident happened:

```
    broken = Workload(**{**AGENT.__dict__, "cache_hit_rate": 0.0})
    rows = [("caching off entirely", uncached(AGENT)),
            ("caching on, 0% hit rate", broken),
            ("caching on, 95% hit rate", AGENT)]
```

```
  configuration               per request    per month  vs no cache
  caching off entirely           0.072750      109,125         0.0%
  caching on, 0% hit rate        0.087750      131,625        20.6%
  caching on, 95% hit rate       0.022200       33,300       -69.5%
  A timestamp at the top of the prompt buys the middle row.
```

The spread between the middle and bottom rows is **$98,325 a month**, and both are "prompt caching is enabled" as far as any configuration review would see. Nothing distinguishes them except `cache_read_input_tokens` in the usage field, which is why section 11.9 insisted you log it.

Testing. Sixteen tests, and the interesting ones validate against something outside the module.

Four assert the published multipliers, so the tests fail if a rate changes:

```
def test_cache_multipliers_match_the_published_ratios():
    """1.25x for a 5m write, 2x for a 1h write, 0.1x for a read."""
    for p in (FABLE_5, OPUS_5, SONNET_5, HAIKU_45):
        assert p.write_5m / p.base_in == pytest.approx(1.25)
        assert p.write_1h / p.base_in == pytest.approx(2.00)
        assert p.cache_read / p.base_in == pytest.approx(0.10)
```

One derives a claim the documentation makes in prose:

```
def test_derived_breakeven_matches_the_documented_claim():
    """The docs say caching pays off after one read (5m) or two (1h).
    We derive it rather than quoting it."""
    for p in (FABLE_5, OPUS_5, SONNET_5, HAIKU_45):
        assert cache_breakeven(p, "5m") == 2     # one write + one read
        assert cache_breakeven(p, "1h") == 3     # one write + two reads
```

One reproduces a worked example printed in the pricing documentation itself, which is the strongest available check that the module is doing the same arithmetic Anthropic is:

```
def test_a_worked_example_from_the_documentation_reproduces():
    """The docs' Managed Agents example: 50k input, 15k output on Opus 5
    is $0.25 + $0.375 in tokens."""
    assert 50_000 * OPUS_5.base_in / MTOK == pytest.approx(0.25)
    assert 15_000 * OPUS_5.out / MTOK == pytest.approx(0.375)
    # And with 40k of the input served from cache, input falls to $0.07.
    cached = 10_000 * OPUS_5.base_in / MTOK + 40_000 * OPUS_5.cache_read / MTOK
    assert cached == pytest.approx(0.07)
```

And two encode the chapter's thesis so that it cannot quietly stop being true:

```
def test_shape_decides_which_line_item_dominates():
    """The chapter's thesis: the lever depends on the workload shape."""
    a = cost_per_request(AGENT, OPUS_5)
    r = cost_per_request(REPORTER, OPUS_5)
    assert max(a, key=a.get) == "prefix"       # caching is the lever
    assert max(r, key=r.get) == "output"       # caching is nearly useless
    assert r["output"] / total(r) > 0.95
def test_caching_with_a_zero_hit_rate_costs_more_than_no_caching():
    broken = Workload(**{**AGENT.__dict__, "cache_hit_rate": 0.0})
    assert monthly(broken, OPUS_5) > monthly(uncached(AGENT), OPUS_5)
```

```
$ cd ch11 && python3 -m pytest -q
................                                                 [100%]
16 passed in 0.13s
```

Deployment. Two homes, as with Chapter 9's calculator. In CI, with your production workload shape as a fixture, failing the build if a proposed prompt change moves projected monthly cost by more than an agreed threshold — this catches the timestamp before it ships rather than after the invoice. And in design review, where the deliverable is the lever table of section 11.8 with your own numbers in it.

Evaluation. Three criteria.

1. **Does it reproduce a bill you have already received?** Feed it last month's actual token counts from your usage logs and compare against the invoice. If it is out by more than a few per cent you have

found an excluded term — most often thinking tokens, retries or conversation growth — and finding it is the point.

2. **Does it distinguish the two rows that look identical?** A model that cannot express "caching enabled, zero hits" cannot warn you about the most expensive configuration mistake in this chapter.
3. **Does anyone bring it to a meeting?** A cost model consulted only by its author is a hobby. The test is whether the next "should we use Opus for this?" conversation ends with a table.

11.13 Hands-on Labs

Lab 11.1 — Price your own workload. Take one real endpoint, pull the token counts from your `usage` logs, and produce the line-item breakdown. *Success criterion: you can state what fraction of the bill is output tokens, and therefore which section of 11.8 applies to you.*

Lab 11.2 — Find your cache hit rate. Compute `cache_read_input_tokens ÷ (input + cache_read + cache_creation)` over a day of traffic. *Success criterion: a number, and — if it is near zero with caching enabled — the specific line of your prompt that is causing it.*

Lab 11.3 — Derive the break-even yourself. Without looking at `cache_breakeven`, work out algebraically how many requests within one cache lifetime make 5-minute caching cheaper, then check against the function. *Success criterion: your algebra and the code agree, and you can explain why the answer is model-independent.*

Lab 11.4 — Measure your latency profile. Instrument TTFT and total latency across a range of prompt and output lengths, and fit the three constants of `LatencyProfile`. *Success criterion: three numbers, and a crossover output length computed from them.*

Lab 11.5 — Build the lever table. For your workload, produce section 11.8's cascade with your own numbers, in the order that applies to your shape. *Success criterion: a table you would be willing to show a finance director, with the quality risk of each row stated.*

11.14 Lab Solutions

11.1. The diagnostic is `output_cost ÷ total`. Above about two thirds, you are generation-bound: cap `max_tokens`, instruct for brevity, and consider a cheaper model before you consider anything else. Below about a third, you are prompt-bound: cache the prefix and verify the hit rate. Between the two, do both, and expect caching to return more per hour of effort because it carries no quality risk.

11.2. A hit rate near zero with caching enabled has three usual causes, in order of frequency: variable content before the cache breakpoint — a timestamp, a request ID, a user name, a randomised example order; a prefix below the minimum cacheable length, which Chapter 4 recorded as 512 tokens on Claude Opus 5, reduced from 1,024 on Opus 4.8; and traffic too sparse to keep a 5-minute cache warm, for which the 1-hour TTL exists at a 2× write premium and a three-request break-even.

11.3. With `n` requests in one lifetime, uncached costs `n·b`. Cached costs `1.25b + (n−1)·0.1b`. Setting the second below the first: `1.25 + 0.1n − 0.1 < n`, so `1.15 < 0.9n`, so `n > 1.28` — meaning `n = 2`. For the 1-hour case, `2 + 0.1n − 0.1 < n` gives `n > 2.11`, so `n = 3`. Both are model-independent because every term is a multiple of `b`, which cancels. That is why the table in section 11.6 shows the same answer for

Opus, Sonnet and Haiku.

11.4. Fit by measuring at the extremes. `ttft_fixed_ms` is TTFT on a minimal prompt. `ttft_per_1k_ms` is the slope of TTFT against prompt length — measure at 1k, 10k and 50k tokens, and expect slight upward curvature from the quadratic of Chapter 4. `tpot_ms` is `(total − TTFT) ÷ output_tokens`, best measured on a long generation where TTFT is a small share. Measure at your real concurrency: all three constants worsen under load, and a profile fitted on an idle system will flatter your p95.

11.5. The order should follow your shape, not this chapter's. Prefix-dominated: caching, then output trimming, then model, then batch. Output-dominated: output trimming, then model, then batch, and caching last because it will return single-digit percentages. State the quality risk on every row that has one — output trimming and model selection both do, and both need Chapter 8's evaluation methodology rather than an assurance.

11.15 Best Practices

BEST PRACTICE

Managing latency, throughput and cost

1. **Log `usage` on every response**, including all three input categories, and derive your cache hit rate from them rather than assuming it.
2. **Order the prompt by rate of change.** Never-changes first, per-session next, per-request last.
3. **Verify the cache is working** before claiming the saving. Zero hits with caching enabled costs 25% more than not caching at all.
4. **Stream to humans, buffer for machines.** It is the cheapest perceived-latency win available and it changes nothing else.
5. **Treat output length as an engineering parameter.** It is the only lever that improves latency, throughput and cost together.
6. **Cap `max_tokens`** to a value your product actually needs — it bounds both the worst-case bill and the worst-case wait.
7. **Batch everything that is not waiting on a human.** A 50% discount for an architectural change you often should have made anyway.
8. **Price agentic workloads per task, not per call**, because one user action becomes many calls with accumulating context.
9. **Include thinking tokens in your output estimate**; they are billed as output and vary per request with adaptive thinking.
10. **Include retries in your cost model.** A retried request is billed twice.
11. **Re-measure token counts after any tokenizer change** with the free `count_tokens` endpoint.
12. **Put a scheduled price change in the model**, not in the invoice — Sonnet 5's introductory rate ends 31 August 2026.
13. **Use `max_content_tokens` on web fetch.** A research-paper PDF is around 125,000 tokens.
14. **Move model selection to the end of the sequence**, after tools and validation have taken over execution, so that you are choosing on the judgement the model actually still does.

11.16 Common Mistakes

COMMON MISTAKE

Enabling prompt caching with variable content in the prefix

Symptom. Caching is enabled, the configuration review passes, and the bill rises by roughly a fifth.

Cause. Causal masking makes caching prefix-only. Any content that changes per request — a timestamp, request ID, user name, randomised example order — invalidates everything after it. Every request becomes a 1.25× write and no request is ever a read.

Fix. Order the prompt by rate of change, with all variable content after the cache breakpoint.

Detection. `cache_read_input_tokens ÷ (input + cache_read + cache_creation)`. If it is near zero, this is what is happening. Nothing else in your telemetry distinguishes the working case from the broken one.

COMMON MISTAKE

Optimising input tokens on an output-dominated workload

Symptom. A sprint spent condensing the system prompt moves the bill by two per cent.

Cause. Output is billed at 5× input on every model. On the report-writer shape in section 11.5, output is 96.2% of cost and the entire prompt is 3.8%. Halving the prompt saves 1.9%.

Fix. Compute the line-item breakdown before choosing what to optimise.

Detection. If you cannot state what fraction of your bill is output tokens, you are guessing about which lever to pull.

COMMON MISTAKE

Treating cached tokens as free of the context budget

Symptom. A prompt engineered around a large cached prefix begins hitting context limits, or shows the attention dilution of Chapter 4 despite "only" a small variable part.

Cause. The documentation is explicit: prompt caching changes what you pay for those tokens, not whether they count toward the window. Cached input still occupies context and still dilutes attention.

Fix. Budget context and cost separately. Caching is a cost lever, not a context lever.

Detection. Compare total input tokens in `usage` against your assumed context consumption. They will match; the caching does not reduce them.

COMMON MISTAKE

Building a cost model on the visible answer length

Symptom. Actual spend on a thinking-enabled workload runs well above the model's prediction, with the gap varying unpredictably by request.

Cause. Thinking tokens are billed as output and are a subset of `max_tokens`. With adaptive thinking the allocation is decided per request, so output tokens are a distribution rather than a constant.

Fix. Take output token counts from `usage`, not from the length of the text you rendered.

Detection. Compare `usage.output_tokens` against the token count of the text you displayed. The gap is the thinking.

COMMON MISTAKE

Assuming latency scales with prompt length

Symptom. Engineering effort goes into prompt shortening to improve responsiveness, and users report no difference.

Cause. Decode dominates. On a cold 12,800-token prompt with a 350-token answer, decode is 81.8% of wall clock; with the prefix cached it is 95.1%. The crossover at which decode overtakes TTFT is 78 output tokens cold and 18 cached.

Fix. Shorten the output, or stream. Prompt length affects TTFT, and TTFT is usually not where the time is.

Detection. Measure TTFT and total separately. If they are close, you are prefill-bound and prompt length matters; if they are far apart, they are not.

COMMON MISTAKE

Reaching for the Batch API where a human is waiting

Symptom. A batch integration is built, and then cannot be used for the traffic that generates most of the cost.

Cause. Batch is asynchronous by definition. Its 50% discount has a hard architectural precondition, and interactive traffic does not meet it.

Fix. Segment traffic by whether anything is waiting on the response. Batch the part that is not — evaluation runs, backfills, overnight classification, regression suites — which is usually a larger fraction than the team assumes.

Detection. Classify a week of requests by whether a human saw the response within a minute. The complement is your batchable share.

11.17 Performance Considerations

Parallelism is the throughput lever you control. You cannot make one response generate faster, because decode is sequential by construction. You can have many requests in flight, bounded by your rate limit tier. For workloads that decompose — classify a thousand documents, evaluate a hundred cases — concurrency converts a linear wait into a bounded one, and it is the difference between a five-hour evaluation run and a ten-minute one.

Time-to-first-token is the number your users judge you on. Everything after the first token is a reading-speed problem for a streaming interface. This is why caching's TTFT benefit matters more than its share of total latency suggests: in the agent scenario it takes perceived response time from 1.40 seconds to 0.32.

Fewer, larger requests usually beat more, smaller ones — up to a point. Each request carries fixed overhead: connection setup, the tool-use system prompt of 286 to 588 tokens, whatever preamble your prompt template adds. Batching ten classifications into one request amortises that. The counterweights are that a failure loses all ten, that the response must all be generated before any of it arrives, and that the accumulating context eventually meets Chapter 4's quadratic.

Agent loops are where cost and latency go to compound. Each turn resends everything so far, so turn `n` costs more than turn `n−1` even for identical work, and total latency is the sum of `n` sequential round trips. The most effective performance intervention available to an agentic system is usually not a faster model but **fewer turns** — better tool descriptions, parallel tool calls, tighter stopping conditions. Chapters 47 to 51 build this properly.

Measure at production concurrency. Every constant in section 11.1's profile degrades under load. A latency budget fitted on an idle system will be met in staging and missed at peak, which is the least useful place to discover it.

11.18 Security Considerations

Cost is an attack surface. An endpoint that accepts arbitrary user text and forwards it to a model with a generous `max_tokens` is a denial-of-wallet target. An attacker does not need to breach anything; they need only send long inputs and elicit long outputs. Validate input length before the model sees it, cap `max_tokens` at a value your product actually needs, and rate-limit per user rather than only in aggregate.

Prompt caching is a shared-prefix mechanism, so scope it by tenant. Chapter 9 made this point from the serving side; the cost model makes it concrete. The economics push you toward putting as much as possible in the stable prefix, and that pressure is exactly what leads someone to put a customer's document there. Cache what is *common by design*, never what is *shared by accident*.

Usage telemetry contains business-sensitive information. Token counts per customer, per feature, per endpoint are a fairly precise map of what your product does and how much of it people use. Treat cost dashboards with the access controls you would apply to revenue data.

Web fetch and web search charge on content you did not choose. Web search is billed per search plus the tokens of what comes back, and web fetch bills the tokens of whatever it retrieves. A prompt-injection payload that induces additional searches or fetches of very large documents is a cost attack as well as a content attack. `max_content_tokens` is a security control, not only a budget control.

Retries amplify everything, including attacks. An aggressive retry policy multiplies both your legitimate bill and the cost of any request designed to be expensive. Bound retries, and bound them per user.

11.19 Real Production Examples

The timestamp. The chapter's opening incident: caching enabled, hit rate zero, bill up 20.6%, cause one line at the top of the system prompt. Detected only because someone eventually looked at `cache_read_input_tokens`. The generalisable control is that a cache hit rate belongs on the same dashboard as latency and error rate, because it is the only signal that distinguishes the expensive misconfiguration from the working one.

The summariser that was billed for its own thinking. A team modelled cost from the length of the summaries they displayed and budgeted accordingly. Actual spend ran roughly double, varying by request. Thinking tokens are billed as output, and with adaptive thinking the allocation is decided per request. The fix was to take output counts from `usage`; the lesson is that the model's cost is what it generated, not what you rendered.

The evaluation run that cost more than the product. A regression suite of several thousand cases, run on every pull request, on the frontier model, synchronously. Nothing was waiting on it — the results were read the next morning. Moving it to the Batch API halved it; moving it to a smaller model for the cases where the frontier model was not the thing under test halved it again. Internal workloads are where batch discounts are most obviously applicable and least often applied.

The agent that was priced per call. A workflow averaging eleven model calls per user action was budgeted at the per-call cost times eleven. Actual cost was roughly three times that, because each call resent the accumulating conversation. Pricing per *task* rather than per *call* made the growth visible, and the fix — compacting tool results out of the context once acted upon, as Chapter 54 describes — took the average back down. Statelessness presents its invoice in agentic systems more than anywhere else.

11.20 Certification Notes

Concept in this chapter	CCAR-F domain	How it is examined
Prompt caching is prefix-only	D5 Context Management and Reliability	Scenario: caching enabled, no saving — identify the variable prefix
Caching accelerates prefill only	D5	Distractor: "caching will speed up generation"
Cache write premiums and read discount	D5, D1	1.25× / 2× / 0.1×, and the break-even that follows
Output tokens cost 5× input	D1 Agentic Architecture	Which lever to pull on a given workload shape
Batch API 50% discount, asynchronous only	D1, D5	Scenario: which traffic qualifies
Streaming changes perceived latency only	D1	Distractor: "streaming reduces cost"
Thinking tokens are billed as output	D4 Prompt Engineering and Structured Output	Cost estimation for thinking-enabled workloads
Long context is standard-priced	D5	Distractor: "1M context carries a premium"
Tool definitions are input tokens every request	D2 Tool Design and MCP Integration	Why tool definitions belong in the cached prefix

Concept in this chapter	CCAR-F domain	How it is examined
Agentic cost grows superlinearly with turns	D1	Scenario: reduce turns, not model size

CERTIFICATION NOTE

The three cost questions that actually appear

"Caching is enabled and costs did not fall." The answer is variable content in the prefix. Look for a timestamp, request ID, session ID, user name or randomised ordering placed before the stable content. The remedy is to reorder by rate of change. A distractor will suggest a longer TTL, which does not help if the prefix never matches.

"Which change reduces cost most for this workload?" Read the shape from the numbers in the stem before reading the options. A large stable prompt with a short answer is prefix-dominated: cache. A short prompt with a long answer is output-dominated: shorten the output or change model, and caching will be a distractor that saves almost nothing.

"Reduce latency for an interactive assistant." Streaming first, because it addresses perceived latency at no cost and no quality risk. Then fewer output tokens. Prompt shortening is a distractor unless the stem tells you TTFT specifically is the problem — decode is typically 80–99% of wall clock.

One trap worth naming: prompt caching appears in both the cost and the latency answers, and it is not equally strong in both. It is a *large* cost lever and a *TTFT-only* latency lever. An option claiming caching speeds up generation is always wrong.

11.21 Practice Questions

1. On the current rate card, output tokens cost how much relative to input, across the model family? A) 2× B) 3× C) 5× D) 10×

2. A 5-minute cache write is billed at what multiple of base input? A) 0.1× B) 1.0× C) 1.25× D) 2.0×

3. A cache read is billed at what multiple of base input? A) 0.1× B) 0.25× C) 0.5× D) 1.0×

4. How many requests within one 5-minute cache lifetime make caching cheaper than not caching? A) 1 B) 2 C) 3 D) 10

5. Caching is enabled and the bill rises 20%. The most likely cause is: A) The TTL is too short B) Variable content precedes the stable prefix C) The model changed D) Output tokens grew

6. Prompt caching improves which of these? A) Time to first token and input cost B) Generation speed C) Output cost D) Context window size

7. A workload sends 1,800 input tokens and generates 4,000 output tokens. The highest-return cost lever is: A) Prompt caching B) Shortening the prompt C) Reducing output length or model tier D) US-only inference

8. The Batch API offers what discount, on what? A) 25% on input only B) 50% on input and output C) 50% on output only D) 90% on cached reads

9. Streaming a response: A) Reduces total generation time B) Reduces cost C) Reduces perceived latency only D) Increases throughput

10. In a request with a 12,800-token prompt and a 350-token answer, decode is approximately what share of wall-clock time? A) 20% B) 50% C) 80% D) 100%

11. Setting `inference_geo: "us"` on a Claude 4.6+ model applies: A) No change B) A 1.1× multiplier on all token categories C) A 50% discount D) A 2× multiplier on output only

12. Per the documentation, the 1M-token context window is priced: A) At a premium above 200K B) At standard rates across the full window C) Only with a beta header D) At double rate

13. Thinking tokens are billed as: A) Input tokens B) Output tokens C) Cache writes D) Not billed

14. Which single lever improves latency, throughput and cost simultaneously? A) Prompt caching B) Generating fewer tokens C) The Batch API D) Fast mode

15. Cached prompt tokens, with respect to the context window: A) Do not count toward it B) Count toward it normally C) Count at 10% D) Extend it

16. Fast mode, per the documentation, is: A) Free B) Priced at $10 input / $50 output on Opus 5 C) Available with the Batch API D) Available on all models

17. Web search is priced at: A) Free B) $10 per 1,000 searches plus tokens C) $1 per search D) Per token only

18. Which is the correct prompt ordering for caching? A) Variable content first B) Alphabetical C) By rate of change, most stable first D) Order is irrelevant

19. In an agentic workflow, cost should be modelled: A) Per API call B) Per task, because context accumulates across turns C) Per user D) Per day

20. The best way to confirm caching is working is: A) Check the configuration B) Compare `cache_read_input_tokens` against total input C) Watch latency D) Trust the feature flag

11.22 Answers and Explanations

1 — C. 5× on every current model: $5/$25 on Opus 5, $2/$10 on Sonnet 5 introductory, $1/$5 on Haiku 4.5, $10/$50 on Fable 5. This ratio is the most useful single fact in the rate card, and it follows from decode being sequential and bandwidth-bound.

2 — C. 1.25× base. **D** is the 1-hour write; **A** is the read. Confusing the two is the difference between a 2-request and a 3-request break-even.

3 — A. 0.1× — a 90% discount on the cached portion.

4 — B. Two: one write at 1.25× plus one read at 0.1× is 1.35, against 2.0 uncached. **C** is the 1-hour answer.

5 — B. The prefix never matches, so every request is a 1.25× write and none is a read. **A** is a plausible distractor and is wrong: a longer TTL cannot help a prefix that differs every time — it would raise the write premium to 2× and make things worse.

6 — A. Caching skips prefill, which reduces time-to-first-token and input cost. **B** is the most common wrong answer and is definitionally wrong — decode is unaffected. **C** is wrong: output is never cached. **D** is wrong: cached tokens still occupy the window.

7 — C. Output is 96.2% of the cost on this shape. **A** saves under 6%, measured. This is the question that separates people who have computed a breakdown from people who have read about caching.

8 — B. 50% on both input and output, in exchange for asynchronous processing.

9 — C. Perceived only. The same tokens are generated and billed, and total generation time is unchanged. It remains the best first change for an interactive product.

10 — C. 81.8% cold, 95.1% with the prefix cached. Decode dominates almost every real request.

11 — B. 1.1× on input, output, cache writes and cache reads alike. Global routing is the default and standard-priced.

12 — B. Standard rates across the full window; a 900,000-token request is billed at the same per-token rate as a 9,000-token one. Long context is expensive because it is many tokens, not because they cost more.

13 — B. Billed as output, counted within `max_tokens`, and counted toward rate limits. With adaptive thinking the amount varies per request.

14 — B. Fewer generated tokens means less decode time, less bandwidth, and less output billing at the 5× rate. Everything else in the list trades one objective against another — batch gives up latency, fast mode gives up cost, caching moves cost and TTFT but not decode.

15 — B. Caching changes what you pay for those tokens, not whether they count. They also still dilute attention.

16 — B. $10 input and $50 output on Opus 5 and Opus 4.8 — double standard rates. It is Claude API only and **not** available with the Batch API, which rules out **C**.

17 — B. $10 per 1,000 searches, plus standard token costs for the content returned. Web *fetch*, by contrast, has no additional charge beyond the tokens of what it retrieves.

18 — C. By rate of change: never-changes, then per-session, then per-request. This is a direct consequence of causal masking.

19 — B. Each turn resends the accumulating conversation, so per-call figures understate the total and understate it more as the task runs longer.

20 — B. Only the usage field distinguishes the working configuration from the broken one. **A** and **D** are exactly what the team in the introduction did.

11.23 Ten Practical Exercises

1. Produce the line-item breakdown for three of your endpoints and rank them by output share.
2. Compute your cache hit rate from a week of usage logs and plot it over time.
3. Reorder one prompt by rate of change and measure the hit rate before and after.

4. Fit your own `LatencyProfile` and compute the crossover output length for two workloads.
5. Classify a week of traffic by whether a human waited for the response; compute your batchable share.
6. Compare `usage.output_tokens` against the rendered text length on a thinking-enabled endpoint and quantify the gap.
7. Model the effect of Sonnet 5's price change on 1 September 2026 across your workloads.
8. Add a CI check that fails if a proposed prompt change moves projected monthly cost by more than 10%.
9. Price one agentic workflow per task rather than per call, and compare against the current budget.
10. Write the one-page cost note you would take to a finance review, with the levers and their quality risks.

11.24 Five Challenge Labs

1. **The full cost surface.** Build a tool that, given a quality requirement and a latency budget, searches over model, output cap, caching configuration and batch eligibility to find the cheapest configuration that satisfies both. Report where the surface is flat and where it is steep.
2. **Cache economics under real traffic.** Simulate your actual arrival process against 5-minute and 1-hour TTLs and find the request rate at which the 1-hour cache's 2× write premium starts paying for itself.
3. **Latency budget allocation.** Given an end-to-end budget of two seconds to first token, allocate it across retrieval, prefill and network, and identify which component to optimise first with measurements rather than intuition.
4. **Denial-of-wallet test.** Design an adversarial input set intended to maximise cost per request against one of your endpoints, and report the worst case you achieve alongside the controls that bound it.
5. **The turn-reduction experiment.** Take an agentic workflow and reduce its average turn count by improving tool descriptions and enabling parallel tool calls. Measure cost, latency and task success before and after, with Chapter 8's sample sizes.

11.25 Interview Questions

1. Why do output tokens cost roughly five times input tokens?
2. Explain prompt caching's pricing to someone who has just heard of it, including where it breaks even.
3. How can enabling prompt caching increase a bill?
4. What fraction of a typical request's latency is decode, and what follows from that?
5. What does streaming actually improve, and what does it not?
6. When would you not use the Batch API?
7. How would you decide between Opus, Sonnet and Haiku for a workload?
8. Why does agentic cost grow faster than the number of calls?
9. What would you log to make cost debuggable?

10. Your bill doubled last month with no traffic change. Walk me through your investigation.

11.26 Summary

Latency has two terms and the second one dominates. Prefill processes the whole prompt in one parallel pass and produces time-to-first-token; decode generates one token at a time and is strictly sequential. On a cold 12,800-token prompt with a 350-token answer, decode is 82% of the wall clock, and once the prefix is cached it is 95%. The crossover at which decode overtakes time-to-first-token is 78 output tokens cold and 18 cached, which essentially every real request exceeds. So latency optimisation means generating fewer tokens, and prompt shortening — the thing that feels like the work — is second-order unless you have measured TTFT specifically and found it large.

The cost equation has the same shape and the same asymmetry: output is billed at five times input on every current model, because a generated token consumes far more of the serving system than a prefilled one. But the equation is trivial and applying it is not, because the right lever depends entirely on the shape of the request. Our support agent sends 12,000 stable tokens and receives 350, so even at a 95% cache hit rate the prefix is its largest line item and caching is transformative. Our report writer sends 1,800 tokens and generates 4,000, so output is 96.2% of its cost and caching saves under 6%. Same company, same rate card, opposite answers — which is why the first question of any cost conversation is not "which model" but "what fraction of the bill is output tokens", and why you answer it from your own `usage` fields rather than from a blog post.

Prompt caching deserves its reputation and its trap in equal measure. The multipliers are constant ratios rather than per-model rates — 1.25× base for a five-minute write, 2× for an hour, 0.1× for a read — so the break-even is model-independent, and we derived it rather than quoting it: two requests within five minutes, three within an hour. That bar is so low that caching is nearly always worth enabling, which means the realistic failure is not neglecting it but breaking it. Because causal masking makes caching prefix-only, one line of variable content near the top of a prompt turns every request into a write and none into a read. Our agent workload went from $109,125 a month uncached to $131,625 with caching "enabled" and broken, to $33,300 with it working. Two of those three configurations pass a configuration review identically. Only `cache_read_input_tokens` tells them apart.

Applied in the right order to a prompt-heavy workload, the levers compound: caching, then output trimming, then model selection, then batch — $109,125 to $8,302 a month, a 92.4% reduction with no change of design. The order matters because caching carries no quality risk and takes an afternoon, while output trimming and model selection both trade against quality and need Chapter 8's methodology rather than an assurance. And the Batch API's 50% is available only where nothing is waiting on a human, a precondition that rules it out for interactive traffic and is met by more internal workloads — evaluation runs, backfills, overnight processing — than most teams notice.

What remains is honesty about what the model excludes, because a cost model that quietly drops a term is worse than none. Thinking tokens are billed as output and vary per request. Retries bill twice. Agentic loops turn one user action into many calls with accumulating context, so the unit of cost is the task and the growth is superlinear. Conversation history grows, which is Chapter 1's statelessness presenting its invoice. And the tokenizer changed at Claude 4.7, producing roughly 30% more tokens for the same text, so any model calibrated before that boundary is wrong after it. All five are visible in `usage` and none is visible in a rate card. Which is the last thing to carry forward: the arithmetic in this chapter is exact, and it

is only ever as good as the two measurements you feed it.

KEY TAKEAWAYS

1. **Latency is TTFT plus output × TPOT**, and decode is 80–99% of it. Generate fewer tokens.
2. **Output costs 5× input** on every current model. Learn the ratio, not the numbers.
3. **The lever depends on the shape.** Ask what fraction of the bill is output before choosing anything.
4. **Cache multipliers are constant ratios**: 1.25× / 2× write, 0.1× read. Break-even is 2 requests (5m) or 3 (1h).
5. **Caching is prefix-only.** Variable content at the top costs 25% more than not caching at all.
6. **Order the prompt by rate of change**, then verify the hit rate from `usage`. Never assume it.
7. **Streaming changes perception, not physics** — and is still the best first change for an interactive product.
8. **Batch is 50% off both directions**, wherever nothing is waiting on a human.
9. **Cached tokens still occupy the context window.** Caching is a cost lever, not a context lever.
10. **Price agentic work per task, not per call**, and include thinking tokens, retries and history growth.

11.27 Glossary

Term	Definition
Batch API	Asynchronous processing at a 50% discount on both input and output tokens.
Cache hit rate	`cache_read_input_tokens` divided by total input tokens; the only signal that caching works.
Cache write premium	1.25× base input for a 5-minute cache, 2× for a 1-hour cache.
Crossover output length	The output length at which decode time overtakes time-to-first-token.
Decode	The sequential, bandwidth-bound phase generating one token at a time.
Denial of wallet	An attack that inflates cost rather than breaching anything.
Fast mode	A research-preview premium tier for faster output on Opus 5 and Opus 4.8, at 2× standard rates.
`inference_geo`	Data-residency parameter; `"us"` applies a 1.1× multiplier to every token category.
MTok	One million tokens, the unit all published rates use.
Prefill	The parallel, compute-bound phase that processes the whole prompt and produces TTFT.
Prompt caching	Reusing a computed prefix to skip prefill; prefix-only, and affects input cost and TTFT only.
Rate of change ordering	Structuring a prompt so the most stable content comes first, for cacheability.
Throughput	Requests completed per unit time; traded against per-request latency.
TPOT	Time per output token; the per-step cost of decode.

Term	Definition
TTFT	Time to first token; the duration of prefill plus request overhead.
`usage` field	Per-response token accounting, split into input, cache read and cache creation.

11.28 Further Reading

Primary sources. Anthropic's *Pricing* page on `platform.claude.com` is the authoritative rate card and the source of every figure in this chapter — the model table, the cache multipliers, the Batch API discount, the fast-mode and data-residency premiums, the tool-use system prompt token counts and the server-tool charges. It was fetched and verified for this chapter, and it is the page to re-check before any cost commitment, because rates and introductory periods change. The *Prompt caching* page covers the implementation, minimum cacheable lengths and breakpoint mechanics; *Batch processing* covers the asynchronous workflow; *Context windows* is the source for cached tokens still counting toward the window; and *Token counting* documents the free `count_tokens` endpoint.

Where to go next in this book. Chapter 12 explains, through scaling laws and benchmarking, why the model tiers are priced as they are and how to read a benchmark before spending money on it. Chapter 13 covers reasoning models and test-time compute — the most direct way to trade money for capability. Chapter 15 turns model selection into a procedure with these costs on one side. Chapter 17 covers streaming as an API surface and Chapter 18 covers thinking, whose tokens this chapter has been pricing. Chapter 19 covers rate limits, which bound the throughput plans here. Chapter 30 is the practical follow-through: batch processing, prompt caching and cost controls as features rather than as economics. And Chapters 52 to 55 address the context growth that makes agentic workloads expensive in the first place.